Machine Coding: Move to Top (Back-to-Top Button)

- **Problem:** On a long page, show a floating "back to top" button that is hidden near the top and fades in after the user scrolls down 400px. Clicking it smoothly scrolls to the top (instantly with reduced motion) and moves keyboard focus to the page heading. It must be a real, accessible button, cheap to run (no heavy scroll handlers), and usable on mobile.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS, `react-icons`)
- **Time limit:** 60 minutes (a small task: expect follow-up questions to fill the time)

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Threshold, motion, focus rules
Use cases + architecture + state	5 min	One boolean: scrolled past
Button + scroll-to-top (simplest)	8 min	Working button
Show/hide on scroll (scroll listener first)	10 min	Appears at the right time
Upgrade to IntersectionObserver; hidden = not focusable	10 min	Efficient + accessible
Reduced motion, focus management, mobile safe areas	7 min	Polished
Test + wrap-up + follow-ups	15 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I don't need the exact scroll position, only one yes/no: is the user past 400px? So instead of a scroll listener firing 60*

times a second, I let an IntersectionObserver tell me when a 1px marker at the top of the page leaves an extended viewport."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A floating button in the bottom-right corner, always above the content.
- Hidden when the page is near the top (< 400px scrolled); fades in after.
- Click (or Enter/Space) → scroll to the top, smoothly.
- Respect `prefers-reduced-motion` (jump instead of animate).
- After scrolling up, keyboard focus is at the top of the page (the main heading).

Non-functional requirements

Concern	What it means here
Performance	No React re-render or JS work per scroll event; IntersectionObserver calls back only when crossing the threshold.
Scalability	Self-contained component + hook; drop it into any page; threshold is a prop.
Error handling	Works if the focus target is missing (`?.`); observer disconnected on unmount.
Accessibility	Real `<button>` with `aria-label` ; icon `aria-hidden` ; hidden state not focusable (`visibility: hidden`); visible focus ring; 48×48px touch target; focus moved to the top heading; reduced motion.
Reusability	`useScrolledPast(offset)` is useful for sticky headers, "show navbar on scroll", etc.
Extensibility	Scroll progress ring, scrolling inside a container, hide while scrolling down.

Clarifying questions to ask the interviewer

1. After how much scrolling should it appear: fixed pixels, one screen height, or a percentage?
2. Smooth scroll or instant? Should it respect reduced motion?
3. Should it scroll the window or a scrollable container (e.g. a chat panel)?
4. Should focus move to the top after scrolling (keyboard / screen-reader users)?
5. Any design: position, show a scroll-progress ring, hide while scrolling down?
6. Must it support old browsers without IntersectionObserver / smooth scroll?

Assumptions we will build with

- Appears after 400px; window scrolling.
- Smooth scroll, or instant with reduced motion.
- Focus moves to the page `<h1>` (`tabIndex={-1}`) without an extra jump.
- Bottom-right, 48px circle, respects mobile safe-area insets.
- Modern browsers (IntersectionObserver and `scrollTo({ behavior })` are widely supported).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page load at the top	Button hidden (not visible, not focusable)
Scroll to 300px	Still hidden
Scroll past 400px	Button fades/slides in
Scroll back above 400px	Button fades out
Click the button	Smooth scroll to the top; focus moves to the `<h1>`; button hides
Tab to the button + Enter/Space	Same as click; visible focus ring
Reduced motion enabled	Instant jump to the top; no fade animation
Reload in the middle of the page	Button is visible immediately (observer's first callback)
Mobile with a home indicator / notch	Button stays inside the safe area
User scrolls during the smooth scroll	Browser hands control back to the user (no fighting)
Screen reader	"Back to top, button"

3. Design the architecture

Module	Responsibility
`hooks/useScrolledPast.js`	`[sentinelRef, scrolledPast]`: IntersectionObserver on a 1px top sentinel with a top `rootMargin` = offset
`components/BackToTop.jsx`	Renders the sentinel + the fixed button; scroll and focus logic on click
`App`	Demo long page: heading (focus target), sections, `<BackToTop offset={400} focusTargetId="top" />`
`App.css`	Fixed position, visible/hidden states, focus ring, safe areas, reduced motion

```
App
├── <h1 id="top" tabIndex={-1}>          ← focus target
├── long <main> content
└── <BackToTop offset={400} focusTargetId="top">
      ├── <div class="top-sentinel" ref>   (absolute, top: 0 of the document)
      │      └── useScrolledPast(400) → IntersectionObserver → scrolledPast (boolean)
      └── <button class="back-to-top visible?" onClick=scrollToTop aria-label="Back to top">
```

Start simple. First version: a fixed button that always shows and calls `window.scrollTo({ top: 0, behavior: "smooth" })`. Then show/hide with a scroll listener (easy to explain), then replace it with IntersectionObserver and add the accessibility details.

4. Design the state / data model

State	Where	Purpose
`scrolledPast`	useScrolledPast	`true` when the page is scrolled more than `offset` px
`sentinelRef`	useScrolledPast	The 1px element the observer watches

```
export function useScrolledPast(offset) {
  const sentinelRef = useRef(null);
  const [scrolledPast, setScrolledPast] = useState(false);
  useEffect(() => {
    const observer = new IntersectionObserver(([entry]) => setScrolledPast(!entry.isIntersecting), {
      rootMargin: `${offset}px 0px 0px 0px`,
    });
    observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [offset]);
  return [sentinelRef, scrolledPast];
}
```

Senior note

- **One boolean, not the scroll position:** storing `scrollY` in state re-renders the component on every scroll event (dozens per second). We only need to know "past the threshold or not", and React re-renders only when that flips.
- **The visible state is CSS, not conditional rendering:** the button stays mounted and toggles a class, so it can fade in/out, and `visibility: hidden` removes it from the Tab order and accessibility tree when hidden.
- **Ownership:** the hook owns *when* to show; the component owns *what happens on click*; the page owns *where focus should go* (`focusTargetId`).

5. Design the data structures and contracts

There's no server data; the contracts are the component API and the hook:

```
<BackToTop
  offset={400}             // px scrolled before the button appears
  focusTargetId="top"      // element to focus after scrolling (should have tabIndex={-1})
/>

const [sentinelRef, scrolledPast] = useScrolledPast(offset);
// sentinelRef → attach to a 1px element at the top of the document
// scrolledPast → boolean
```

Why a sentinel + rootMargin works:

```
rootMargin "400px 0 0 0" grows the viewport 400px upwards.
Sentinel at document top (y = 0).
Scrolled 300px → sentinel is 300px above the screen → inside the grown root → intersecting → hide.
Scrolled 500px → sentinel is 500px above the screen → outside → not intersecting → show.
```

Browser APIs used: `IntersectionObserver`, `window.scrollTo({ top, behavior })`, `matchMedia("(prefers-reduced-motion: reduce)")`, `element.focus({ preventScroll: true })`, CSS `env(safe-area-inset-*)`.

6. Trace the data flow

User scrolls down, then clicks the button:

```
Scroll 0 → 900px
  ↓ sentinel leaves the (grown) root at 400px → observer callback → setScrolledPast(true)   (one call, not 60/s)
  ↓ React re-renders BackToTop → class "visible" → CSS fades in, visibility visible (focusable)
Click "Back to top"
  ↓ reduceMotion? (matchMedia) → no
  ↓ document.getElementById("top").focus({ preventScroll: true })   (focus without a jump)
  ↓ window.scrollTo({ top: 0, behavior: "smooth" })                  (browser animates)
  ↓ sentinel re-enters the root at 400px → callback → setScrolledPast(false)
  ↓ class removed → fade out → visibility hidden after 0.2 s (removed from Tab order)
```

7. Important technical challenges

Challenge 1: knowing when to show the button, cheaply

- *Problem:* A naive `window.addEventListener("scroll", () => setY(window.scrollY))` fires dozens of times per second and re-renders React each time; on slow phones scrolling becomes janky. Forgetting `removeEventListener` leaks the listener.
- *Solution:* IntersectionObserver on a 1px sentinel with `rootMargin` = threshold. The callback runs only when the threshold is crossed, off the scroll hot path. (If you use a scroll listener anyway: `{ passive: true }`, rAF throttling, store only the boolean, clean up.)

```
const observer = new IntersectionObserver(([entry]) => setScrolledPast(!entry.isIntersecting), {
  rootMargin: `${offset}px 0px 0px 0px`,
});
observer.observe(sentinel);
return () => observer.disconnect();
```

Challenge 2: a real, accessible button

- *Problem:* The first version made an SVG icon clickable (`<FaArrowUp onClick>`). An SVG isn't focusable or announced as a button, so keyboard and screen-reader users can't use it. It was also placed `absolute` inside the last section, so it only existed at the bottom of the page.

- *Solution:* A `<button>` with an accessible name, the icon hidden from assistive tech, fixed positioning, a 48px target and a visible focus ring.

```
<button type="button" className={`back-to-top ${visible ? "visible" : ""}`}
        onClick={scrollToTop} aria-label="Back to top" title="Back to top">
  <FaArrowUp aria-hidden="true" />
</button>
```

Challenge 3: hidden must mean hidden for everyone

- *Problem:* Hiding with only `opacity: 0` leaves an invisible button that keyboard users can still Tab to (a "ghost" focus stop) and screen readers still read. Unmounting it (`{visible && <button>}`) kills the fade-out animation.
- *Solution:* Keep it mounted and use `visibility: hidden` (removes it from focus and the accessibility tree) with a delayed transition so the fade still plays.

```
.back-to-top { opacity: 0; visibility: hidden; transform: translateY(12px);
  transition: opacity .2s, transform .2s, visibility 0s linear .2s; }   /* hide visibility AFTER the
fade */
.back-to-top.visible { opacity: 1; visibility: visible; transform: none;
  transition: opacity .2s, transform .2s; }                              /* show visibility immediately
*/
```

Challenge 4: scrolling up properly (motion + focus)

- *Problem:* Only scrolling moves the *view*, not keyboard *focus*: focus stays on the button at the bottom, so the next Tab jumps back down. Smooth scrolling can make some users dizzy (vestibular disorders). The first version also had a typo in the commented code (`scrollBehavior: "'smooth',"` instead of `behavior: "smooth"`).
- *Solution:* Focus the top heading without an extra jump, then scroll; skip the animation for reduced motion.

```
const reduceMotion = window.matchMedia("(prefers-reduced-motion: reduce)").matches;
document.getElementById(focusTargetId)?.focus({ preventScroll: true });
window.scrollTo({ top: 0, behavior: reduceMotion ? "auto" : "smooth" });
```

8. Implement in the correct order

#	Step	Working result
1	Long demo page with sections	Something to scroll
2	Fixed `<button>` + `window.scrollTo({ top: 0, behavior: "smooth" })`	Works (always visible)
3	Show/hide with a scroll listener + boolean state (+ cleanup)	Correct visibility
4	Replace with `useScrolledPast` (IntersectionObserver + sentinel)	Efficient
5	CSS visible/hidden with `visibility` + delayed transition	Fades, no ghost focus

#	Step	Working result
6	`aria-label`, `aria-hidden` icon, focus ring, 48px target	Accessible
7	Focus target (`<h1 tabIndex={-1}>`) + `focus({ preventScroll: true })`	Keyboard users land at the top
8	`prefers-reduced-motion` in JS (scroll) and CSS (fade)	Motion-safe
9	`env(safe-area-inset-*)`, z-index	Mobile-friendly
10	Test: thresholds, Tab order, click, reload mid-page	Verified

9. Cover all states and edge cases

State / edge case	Handling
At the top	Hidden, not focusable
Just below the threshold (300px)	Hidden
Past the threshold	Visible, focusable
Reload mid-page / deep link to `#section-8`	Observer's first callback reports "not intersecting" → visible
Page shorter than the threshold	Never appears
Click while already near the top	Not possible (hidden)
User scrolls during the smooth scroll	Browser stops the animation and follows the user
Reduced motion	Instant jump, no fade
Focus target missing	`?.` → just scrolls
Unmount	Observer disconnected
Mobile notch / home bar	`env(safe-area-inset-*)` offsets
Content under the button	z-index above content; small footprint in the corner
Very long page with lazy content	Unaffected (the sentinel is at the document top)

10. Final structure

```
_13_move_to_top/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css                ← global reset
    ├── App.jsx                  ← demo long page + <BackToTop />
    ├── App.css                  ← button states, focus, safe areas, reduced motion
    ├── hooks/
    │   └── useScrolledPast.js   ← IntersectionObserver + sentinel
    └── components/
        └── BackToTop.jsx        ← sentinel + fixed button + scroll/focus logic
```

11. Full working code

`src/hooks/useScrolledPast.js`

```
import { useEffect, useRef, useState } from "react";

// true once the page is scrolled more than `offset` px. Uses IntersectionObserver on a
// 1px sentinel at the very top instead of a scroll listener: no work on every scroll event.
export function useScrolledPast(offset) {
  const sentinelRef = useRef(null);
  const [scrolledPast, setScrolledPast] = useState(false);

  useEffect(() => {
    const sentinel = sentinelRef.current;
    if (!sentinel) return;
    // A positive top rootMargin grows the viewport upwards by `offset`, so the sentinel
    // counts as "visible" until it is more than `offset` px above the screen.
    const observer = new IntersectionObserver(([entry]) => setScrolledPast(!entry.isIntersecting), {
      rootMargin: `${offset}px 0px 0px 0px`,
    });
    observer.observe(sentinel);
    return () => observer.disconnect();
  }, [offset]);

  return [sentinelRef, scrolledPast];
}
```

`src/components/BackToTop.jsx`

```
import { FaArrowUp } from "react-icons/fa";
import { useScrolledPast } from "../hooks/useScrolledPast";

// Floating "back to top" button: hidden near the top, appears after `offset` px of scrolling.
const BackToTop = ({ offset = 400, focusTargetId }) => {
  const [sentinelRef, visible] = useScrolledPast(offset);

  const scrollToTop = () => {
    const reduceMotion = window.matchMedia("(prefers-reduced-motion: reduce)").matches;
    // Move keyboard / screen-reader focus to the top too, otherwise the next Tab jumps back down.
    if (focusTargetId) document.getElementById(focusTargetId)?.focus({ preventScroll: true });
    window.scrollTo({ top: 0, behavior: reduceMotion ? "auto" : "smooth" });
  };

  return (
    <>
      {/* position: absolute with no positioned ancestor = pinned to the top of the document */}
      <div ref={sentinelRef} className="top-sentinel" aria-hidden="true" />
```

```
      <button
        type="button"
        className={`back-to-top ${visible ? "visible" : ""}`}
        onClick={scrollToTop}
        aria-label="Back to top"
        title="Back to top"
      >
        <FaArrowUp aria-hidden="true" />
      </button>
    </>
  );
};

export default BackToTop;
```

src/App.jsx

```
import "./App.css";
import BackToTop from "./components/BackToTop";

const SECTIONS = Array.from({ length: 12 }, (_, i) => ({
  id: `section-${i + 1}`,
  title: `Section ${i + 1}`,
}));

function App() {
  return (
    <>
      <header className="page-header">
        {/* tabIndex -1: focusable by script (BackToTop), not part of the Tab order */}
        <h1 id="top" tabIndex={-1}>
          Move to Top
        </h1>
        <p>Scroll down; the button appears after 400px. Click it (or press Enter on it) to go back up.
</p>
      </header>

      <main className="content">
        {SECTIONS.map((section) => (
          <section key={section.id} id={section.id} aria-labelledby={`${section.id}-title`}>
            <h2 id={`${section.id}-title`}>{section.title}</h2>
            <p>
              Long pages need a quick way back to the navigation. This paragraph is filler content so
the page is
              tall enough to scroll. Notice the button stays out of the way at the top and fades in once
you have
              scrolled.
```

```
            </p>
            <a href={`#${section.id}`}>Link in {section.title}</a>
          </section>
        ))}
      </main>

      <BackToTop offset={400} focusTargetId="top" />
    </>
  );
}

export default App;
```

src/App.css

```
body { background: #f9fafb; color: #111827; font-family: system-ui, sans-serif; line-height: 1.6; }
.page-header { text-align: center; padding: 48px 16px 24px; background: #eef2ff; }
.page-header h1 { font-size: 36px; }
.page-header h1:focus { outline: none; }
.content { max-width: 680px; margin: 0 auto; padding: 16px; }
section { min-height: 60vh; padding: 24px 0; border-bottom: 1px solid #e5e7eb; }
section h2 { margin-bottom: 8px; }
a { color: #4338ca; }

.top-sentinel { position: absolute; top: 0; left: 0; width: 1px; height: 1px; pointer-events: none; }

.back-to-top {
  position: fixed;
  right: max(16px, env(safe-area-inset-right));
  bottom: max(16px, env(safe-area-inset-bottom));
  z-index: 100;
  width: 48px; height: 48px;                /* comfortable touch target */
  display: grid; place-items: center;
  border: none; border-radius: 50%;
  background: #4f46e5; color: white; font-size: 18px;
  box-shadow: 0 6px 16px rgba(79, 70, 229, 0.35);
  cursor: pointer;
  /* Hidden state: invisible AND not focusable (visibility removes it from the Tab order). */
  opacity: 0; visibility: hidden; transform: translateY(12px);
  transition: opacity 0.2s, transform 0.2s, visibility 0s linear 0.2s;
}
.back-to-top.visible { opacity: 1; visibility: visible; transform: none; transition: opacity 0.2s,
transform 0.2s; }
.back-to-top:hover { background: #4338ca; }
.back-to-top:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 3px; }

@media (prefers-reduced-motion: reduce) {
```

```
  .back-to-top, .back-to-top.visible { transition: none; }
}
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: threshold detection without scroll listeners, real button semantics, hidden = not focusable, fade in/out without unmounting, reduced motion (JS and CSS), focus moved to the top, deep links / reload mid-page, safe areas, observer cleanup.

Trade-offs to mention to the interviewer

- *IntersectionObserver vs scroll listener:* the observer is cheaper and simpler for a yes/no threshold. A scroll listener is needed when you want the exact position (progress ring, parallax); then use `{ passive: true }` + rAF throttling.
- *Window vs container scrolling:* this version targets the window. For a scrollable panel, pass the container as the observer `root` and call `container.scrollTo`.
- *Smooth scroll in JS vs CSS* `scroll-behavior: smooth` *:* the CSS rule affects every anchor jump on the page (sometimes unwanted); the JS option scopes it to this button and lets us check reduced motion.
- *Moving focus:* better for keyboard and screen-reader users, but it must not cause a second jump (`preventScroll`) or show an odd focus ring on the heading (`:focus { outline: none }` on a non-interactive target is acceptable since it's not in the Tab order).

What to add with more time

- Scroll-progress ring around the button (SVG `stroke-dashoffset` from `scrollY / (scrollHeight - innerHeight)` , rAF-throttled).

- Hide while scrolling down, show while scrolling up (common on mobile).
- Support scroll containers via a `rootRef` prop.
- Tests: mock IntersectionObserver to fire `isIntersecting` true/false and assert the class; assert `scrollTo` is called with `behavior: "smooth"` or `"auto"` (mocked `matchMedia`); assert focus moves to `#top`.

13. Counter questions the interviewer may ask

Requirements and design

Q1. When does a back-to-top button actually help?

On long pages (articles, feeds, documentation, long product lists) where scrolling back is tedious, especially on mobile. On short pages it's clutter. Some designers prefer a sticky header instead; ask what problem we're solving.

Q2. Fixed pixels, one viewport, or a percentage as the threshold?

Pixels are simple and predictable. One viewport height (`innerHeight`) adapts to screen size ("you've scrolled a full screen"). A percentage fails on very long pages (10 % could be 5,000px). I used a prop, so the page decides.

Q3. Should the button hide while the user scrolls down?

Many mobile UIs show it only while scrolling **up** (the user is heading back) to save space. That needs the scroll direction, so a throttled scroll listener comparing the last `scrollY`.

State design

Q4. Why store a boolean instead of `scrollY`?

The UI only changes when the threshold is crossed. Storing `scrollY` would re-render on every scroll event for nothing. Store the smallest piece of state that drives the UI.

Q5. Why keep the button mounted instead of `{visible && <button/>}`?

Unmounting removes it instantly, so there's no fade-out, and the fade-in needs extra tricks. Keeping it mounted and toggling a class allows CSS transitions; `visibility: hidden` still removes it from the Tab order and accessibility tree.

Q6. Where does the "focus target" come from, and why `tabIndex={-1}`?

The page knows what "top" means (usually the main heading or a skip-link target), so it passes `focusTargetId`. `tabIndex={-1}` makes a non-interactive element focusable by script without adding it to the Tab order.

Hard parts

Q7. Explain the IntersectionObserver `rootMargin` trick.

`rootMargin: "400px 0 0 0"` extends the observed area 400px above the viewport. A 1px sentinel at the document top stays "intersecting" until it is more than 400px above the screen. So `!isIntersecting` means

"scrolled past 400px", with no scroll handler at all.

Q8. If you used a scroll listener, how would you make it efficient?

`addEventListener("scroll", handler, { passive: true })` (tells the browser we won't call `preventDefault`, so scrolling isn't blocked), throttle with `requestAnimationFrame` (at most once per frame), read `scrollY` only, set state only when the boolean changes, and remove the listener in cleanup.

Q9. `window.scrollTo` vs `element.scrollIntoView` vs CSS `scroll-behavior`?

`scrollTo({ top: 0, behavior })` scrolls the window to a position. `scrollIntoView({ behavior })` scrolls whatever is needed to show an element (works for nested containers; the first version used it on the first section). `html { scroll-behavior: smooth }` makes **all** scrolls smooth, including anchor links and `scrollTo` without a behavior. Pick the narrowest one.

Q10. Why move focus after scrolling to the top?

Scrolling moves what you see, not where keyboard focus is. Without moving focus, a keyboard user who presses Tab after "Back to top" jumps right back down to the button. Screen-reader users also get no feedback. Focusing the heading puts them at the top in both senses.

Q11. How would you make it work for a scrollable container (a chat panel)?

Pass the container element as the observer's `root` (with the same `rootMargin`), place the sentinel at the top **inside** the container, and call `container.scrollTo({ top: 0, behavior })`. The button can stay fixed inside the container's positioned wrapper.

Implementation steps

Q12. How do you test this?

jsdom has no layout or IntersectionObserver, so mock `IntersectionObserver` (capture the callback, call it with `{ isIntersecting: false }`) and assert the button gets the `visible` class. Mock `window.scrollTo` and `matchMedia` to check `behavior: "smooth"` vs `"auto"`, and check that `#top` receives focus. Use Playwright for the real scroll behaviour.

Q13. How do you add a scroll-progress ring around the button?

Progress = `scrollY / (scrollHeight - innerHeight)`, computed in a passive, rAF-throttled scroll listener. Draw an SVG circle with `stroke-dasharray = circumference` and `stroke-dashoffset = circumference * (1 - progress)`. Update via a CSS variable or a ref to avoid re-rendering React on every frame.

Q14. What's the problem with the first version's `scrollBehavior: "'smooth',"`?

The option name is `behavior`, not `scrollBehavior` (that's the CSS property name), and the value was the string `'smooth',` with extra quotes and a comma. Unknown options are silently ignored, so it would jump instead of animating. Typos in option objects fail silently; read the API carefully.

Performance and scale

Q15. Why is a non-passive scroll listener bad?

The browser must wait for your handler to finish before scrolling, in case it calls `preventDefault()`. A slow handler then makes scrolling stutter. `{ passive: true }` promises you won't, so the browser scrolls immediately (and Chrome warns about non-passive listeners on touch/wheel).

Q16. Many components need "has the user scrolled?" Should each one observe?

One IntersectionObserver instance can watch many elements, but here many components want the same boolean. Put it in a context or a small shared store (`useSyncExternalStore`) with one observer, and let components subscribe.

Q17. Could the fade animation hurt performance?

No: it animates `opacity` and `transform`, which the compositor handles without layout or paint. Avoid animating `bottom`, `width` or `box-shadow` on scroll-linked elements.

Trade-offs and "why not X?"

Q18. Why not CSS-only (`position: sticky` or scroll-driven animations)?

CSS scroll-driven animations (`animation-timeline: scroll()`) can fade a button in based on scroll position with no JS, and they are great where supported. But browser support is newer, and you still need JS for focus management. Feature-detect and enhance.

Q19. Why not a library?

It's ~40 lines with two browser APIs; a dependency adds weight and less control. Libraries make sense for complex scroll features (scroll spy, virtualisation).

Q20. Why not put `scroll-behavior: smooth` on `html` and use an `<a href="#top">` link?

That's a valid no-JS option: an anchor link to the top works without JavaScript and moves focus to the target in many browsers. But global smooth scrolling affects every jump on the page, `#top` changes the URL hash (Back-button entries), and you still need JS to show/hide the button.

Q21. What about users who navigate only with the keyboard: is the floating button enough?

Keyboard users usually press Home (or Ctrl+Home) to go to the top, so the button mostly helps mouse/touch users. Still, it must be focusable and labelled when visible. A "skip to top" link in the footer is a good complement.